HW2

Problem 1

1. 2 queue → 1 stack

1) push(). Just enqueue() to queue 1. So the running time should be $O(1)$

2) pop(). Dequeue() queue 1 and enqueue() to queue 2 until there is only 1 element in queue 1. Dequeue the last element of queue 1. and that is the result of pop(). Swap the name of queue 1 and queue 2. return the value we get. So every operation of pop() will go through the whole queue so the running time should be $O(n)$

2. 2 stack → 1 queue

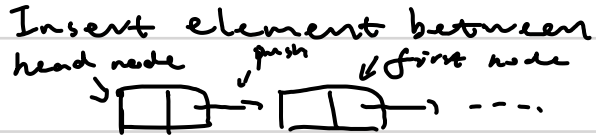
1) enqueue() Just push to stack 1. So the time consuming should be $O(1)$.

2) dequeue() Just pop and push every element from stack 1 to stack 2 except the last element. Then pop stack 1 and swap the name of stack 1 and 2. Return the value we get. Same reason above the running time should be $O(n)$.

Problem 2

1. Stack.

1) push. Insert element between the head node and the first node.



the algorithm will get the head node pointer

2) pop(). Return the value of the head node. and head = head.next

2. queue

1) enqueue(). By setting an pointer last to the last element. Every time a new element going in. last.next = New_Insert_Node New_Insert_Node.next = Null last = last.next

2) dequeue(). Return the first elements of linked-list.

3. Union . Make the linked-list with the given disjoint set. Maintain a front and back pointer to the first and last element. Since it's two disjoint set. There's no duplicate element. So just make S.back.next = T.start.

Problem 3

1. (u,v) is a forward edge which means that v is a deeper node which means d(u) < d(v), v has to be finish iteration and return and when conditions satisfied u finished. So we got d(u) < d(v) < f(v) < f(u).

2. A back edge (u,v) means v is the ancestor of u, complete opsite of (1.), so d(v) < d(u) < f(u) < f(v) proved.

3. cross edge (u,v), they got connected but do not share any ancestor and decentant, since it point from u to v, so v fast to be discoved and finished first otherwised if u got discoverd it there's a edge so v would be its discendant it conflict the defination. so d(v) < f(v) < d(u) < d(u)

Problem 4. for each v in V: indegree[v] = 0

1. for each edge(src, dest) in Edge:

indgree[dest] ++

while S != NULL: $O(V)$

pop element from S:

add u to L (result) log V

decrease all the node neighbouring nodes in-degree and Edges $O(m)$ total

add all in-degree 0 nodes to S $O(n+m)$

if L.size < V.size: there's at least 1 loop fail

else: return L.

∴ Proved this algorithm runs at $O(n+m)$

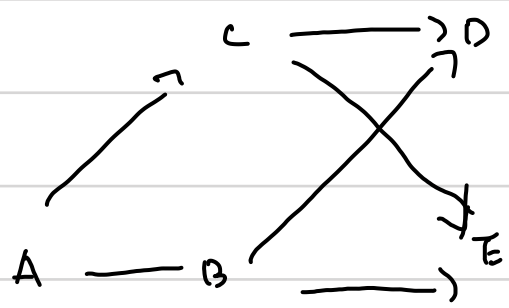
2. if there's a loop, there are always a Path (u,v), with in-degree 1 which means not included during the algorithm

· Since the Outer Loop running time's $O(n)$, inner Loop running time based on the number of edge. over all will be $O(m)$, plus the part of find in-degree nodes, so $O(n+m)$ proved

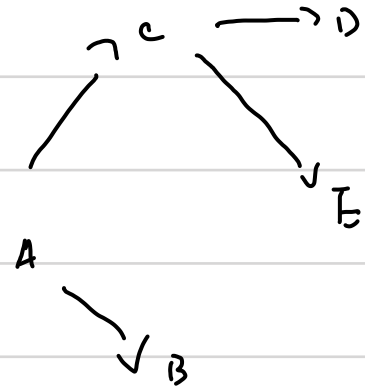
3. ~~since~~ Based on Algorithm, there's no nodes satisfying the conditions after we go through other nodes. The loop. if we use V_1 as example. it will have a at least in-dgree of 1, since V_n pointing to it, since the sorted nodes number are smaller than V, it's proofed that we have a loop.

Problem 5:

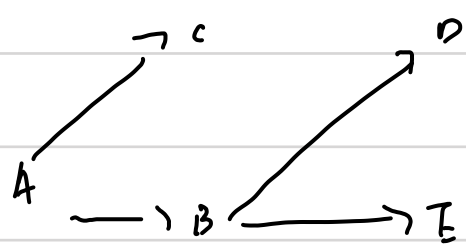
As we all know, if a node discovered, there's no other tree edge to that node in BFS. So based on that we need create a multiplue start point Graph.



so start with C.



Start with B



so if we choose

(A, B) (A, C) (C, D) (B, E)

It will not be satisfied by BFS.

Problem 6:

```
GCDE(a, b, x, y)
1.  if (a=0):
        x=0   y=1   return b
    int x1=1   y1=1   gcd=gcd(b%a, a, x1, y
    x=y1-(b/a)*x1
    y=x1
    return gcd
```

the running time's $O(\log \min(a, b))$, since it's recursive it should be log based, the smaller number of a and b determine how depth of the recersive will go.

2. we can simplely find gcd of (a, m), if gcd != 1, break

else inverse = (x % m + m) % m;

↑

handle

~~prevent~~ negative x

example $75 = 3 \times 28 + 19$

$28 = 1 \times 19 + 9$

$19 = 2 \times 9 + 1$

$\therefore 1 = 19 - 2 \times 9$

$\therefore 1 = 19 - 2 \times (28 - 19)$

$= -2 \times 28 + 3 \times 19$

$= -2 \times 28 + 3 \times (75 - 2 \times 28)$

$= -8 \times 28 + 3 \times 75$

$\therefore x = -8 \quad \therefore$ inverse $= 67$